

Zero to Hero Study Handbook: rag-telemetry-evals-ollama

Module 1: Foundations & Architecture

1.1 What this project does

This repository builds a **local Retrieval-Augmented Generation (RAG)** system on top of Ollama, then measures quality and runtime behavior with persisted artifacts.

Primary goals implemented in code:

- Build a local vector index from markdown/text knowledge files.
- Answer questions in two modes:
 - **Baseline** (no retrieval): `LocalRAGEngine.answer_without_rag(...)`
 - **RAG** (retrieval + generation): `LocalRAGEngine.answer_with_rag(...)`
- Evaluate both modes across a labeled question set (`data/eval/questions.json`).
- Save reproducible outputs:
 - predictions CSV
 - summary metrics JSON
 - markdown report
 - raw telemetry traces JSONL
 - aggregated telemetry summary JSON

1.2 Main use cases

- Learning a complete local RAG pipeline (indexing -> retrieval -> answer generation).
- Observing model/runtime behavior with structured traces (`JsonlTelemetryTracer`).
- Comparing baseline vs RAG quality with explicit metrics (`keyword_recall`, `semantic_similarity`).
- Producing portfolio artifacts from one command (`rag-telemetry-evals run-all`).

1.3 Core paradigms and patterns used here

Definitions first, then where they appear in this repo:

1. Layered architecture

- Definition: separate responsibilities into layers (configuration, data loading, retrieval, model gateway, evaluation, reporting).
- In this repo: `config.py` -> `pipeline.py` orchestrates -> domain modules under `data/`, `retrieval/`, `eval/`, `telemetry/`, `reporting/`.

2. Typed data modeling

- Definition: enforce structured inputs/outputs with explicit schemas.

- In this repo: Pydantic models in `schemas.py` (`Document`, `DocumentChunk`, `QAExample`, `EvalPrediction`, `EvalSummary`, `TraceSpanRecord`) and `RetrievedChunk` dataclass.

3. Async I/O workflow

- Definition: non-blocking execution for external calls (model list/chat/embed).
- In this repo: `AsyncOllamaGateway` methods are async; orchestration functions (`build_index`, `answer_question`, `evaluate`, `run_all`) are async.

4. Pipeline orchestration

- Definition: deterministic sequence of stages where each stage emits artifacts.
- In this repo: `pipeline.py` sequences indexing, QA, evaluation, trace summarization, and report rendering.

5. Context-manager telemetry spans

- Definition: instrument stages with start/end timing and attributes.
- In this repo: with `tracer.span(trace_id, span_name, attributes)` in `pipeline.py` and `rag_engine.py`, persisted to JSONL.

6. Fallback strategy pattern

- Definition: degrade gracefully when primary path fails.
- In this repo:
 - retrieval fallback: vector -> keyword overlap (`_keyword_fallback`) on embedding timeout.
 - generation fallback: extractive text or conservative answer on chat timeout.

1.4 Architecture and component interaction

Key runtime components:

- **CLI Layer:** `src/rag_telemetry_evals/cli.py`
- **Settings Layer:** `src/rag_telemetry_evals/config.py`
- **Pipeline Orchestration:** `src/rag_telemetry_evals/pipeline.py`
- **Data Loading + Chunking:** `data/documents.py`, `retrieval/chunking.py`
- **Indexing + Search:** `retrieval/vector_index.py`
- **Model Gateway:** `ollama_client.py`
- **RAG Engine:** `retrieval/rag_engine.py`
- **Evaluation:** `eval/evaluator.py`
- **Telemetry:** `telemetry/tracer.py`
- **Reporting:** `reporting/reporting.py`

ASCII main flow:

```
User CLI Command (Typer)
  |
  v
```

```

get_settings() + ensure_dirs()
|
+--> build-index -----+
|
|   load_documents() -> chunk_documents()
|       -> embed_texts() -> VectorIndex
|       -> save_vector_index()
|       -> artifacts/index/{embeddings.npy,chunks.json}
|
+--> ask (--rag) -----+
|
|   load_index() -> LocalRAGEngine
|   retrieve() [vector or keyword fallback]
|   chat() [or timeout fallback answer]
|   -> JSON payload with retrieved chunks
|
+--> ask (--baseline) -----+
|
|   LocalRAGEngine.answer_without_rag()
|   chat() [or timeout fallback "I do not know."]
|
+--> evaluate -----+
|
|   load_eval_examples()
|   run_evaluation() per question
|   save_predictions() + save_summary()
|   summarize_traces() + render_report()
|   -> artifacts/evals, artifacts/reports,
|       artifacts/telemetry
|
+--> run-all = build-index + evaluate + run_summary.json

```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
README.md	User-facing project overview and command sequence	n/a	model names, setup/run commands
pyproject.toml	Packaging, deps, tool configs, CLI entrypoint	[project.scripts]rag-telemetry-eval	runtime deps (ollama, pydantic, numpy, typer, etc.), requires-python

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>.env.example</code>	Environment variable template	n/a	OLLAMA_HOST, CHAT_MODEL, EMBEDDING_MODEL, chunk/retrieval/generation/path keys
<code>src/rag_telemetry/telemetry_cli.py</code>	Telemetry CLI commands and async dispatch	<code>_run_async</code> , <code>build_index_cmd</code> , <code>ask_cmd</code> , <code>evaluate_cmd</code> , <code>run_all_cmd</code> , <code>summarize_telemetry_cmd</code>	command options <code>--rag/--baseline</code>
<code>src/rag_telemetry/telemetry_configs.py</code>	Defaults, configs and resolved artifact paths	<code>Settings</code> , <code>get_settings</code> , <code>ensure_dirs</code>	<code>chunk_size_words</code> , <code>retrieval_top_k</code> , <code>generation_timeout_seconds</code> , <code>embedding_timeout_seconds</code> , path properties
<code>src/rag_telemetry/telemetry_pipeline.py</code>	Telemetry pipeline orchestration	<code>build_index</code> , <code>load_index</code> , <code>answer_question</code> , <code>evaluate</code> , <code>run_all</code>	uses settings-derived paths and models
<code>src/rag_telemetry/telemetry_schemas.py</code>	Telemetry schemas for data and outputs	<code>Document</code> , <code>DocumentChunk</code> , <code>QAExample</code> , <code>ChatResult</code> , <code>TraceSpanRecord</code> , <code>EvalPrediction</code> , <code>EvalSummary</code>	strict schema validation (<code>extra="forbid"</code>)
<code>src/rag_telemetry/telemetry_ollama_client.py</code>	Async wrapper around Ollama client	<code>AsyncOllamaGateway</code> , <code>ensure_required_model_names</code> , <code>embed_texts</code> , <code>chat</code>	<code>chat_options</code> , <code>temperature</code> , <code>num_predict</code> , timeout handling
<code>src/rag_telemetry/telemetry_data_documents.py</code>	Load data/documents knowledge docs from disk	<code>load_documents</code> , <code>_extract_title</code>	<code>SUPPORTED_SUFFIXES={" .md", ".txt" }</code>
<code>src/rag_telemetry/telemetry_vector_retrieval/chunking.py</code>	Vector retrieval chunking with overlap	<code>chunk_documents</code>	<code>chunk_size_words</code> , <code>chunk_overlap_words</code>
<code>src/rag_telemetry/telemetry_vector_retrieval/vector_index.py</code>	Vector retrieval build/search/persist/load	<code>VectorIndex</code> , <code>build_vector_index</code> , <code>save_vector_index</code> , <code>load_vector_index</code>	cosine similarity, <code>top_k</code> , <code>.npy</code> + <code>chunks.json</code>

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/rag_telemetry/evals/retrieval/legals/rag_engine.py	Relevant retrieval generation logic with fallbacks	<code>LegalRAGEngine</code> , <code>answer_with_rag</code> , <code>answer_without_rag</code> , <code>_keyword_fallback</code>	<code>RAG_SYSTEM_PROMPT</code> , <code>BASELINE_SYSTEM_PROMPT</code> , <code>retrieval/generation timeouts</code>
src/rag_telemetry/evals/eval/rag_evaluator.py	Baseline eval/AG scoring and summary metrics	<code>LegalRAGEvaluator</code> , <code>run_evaluation</code> , <code>_semantic_similarity</code>	<code>required_keywords</code> , <code>expected_source</code> , <code>embedding</code> , <code>timeout capped at 8s in scorer</code>
src/rag_telemetry/evals/telemetry/trace.py	Spans/telemetry and aggregation	<code>Trace</code> , <code>summarize_traces</code>	<code>traces.JSONL</code> , <code>fields: trace_id, span_name, status, latency_ms, attributes</code>
src/rag_telemetry/evals/reporting/save_summary.py	Save JSON and render markdown report	<code>save_summary</code> , <code>render_report</code>	<code>REPORTING_OPTIONS</code> , <code>_REPORT_TEMPLATE</code> , <code>Jinja2 markdown template</code>
scripts/run_pipeline.py	Snappy convenience wrapper for <code>run_all</code>	<code>_main</code>	prints full payload JSON
scripts/execute_notebooks.py	Execute tutorial notebooks in fixed order	<code>execute_notebook</code> , <code>NOTEBOOKS</code>	notebook timeout 1800 seconds
scripts/generate_notebooks.py	Generate tutorial notebooks	<code>_write_notebook</code> , <code>main</code>	notebook names and source cells
data/eval/question_examples.json	Eval json dataset	JSON list mapped to <code>QAExample</code>	keys: <code>question_id</code> , <code>question</code> , <code>reference_answer</code> , <code>expected_source</code> , <code>required_keywords</code>
data/knowledge/	Local corpus used for indexing/retrieval	markdown docs consumed by <code>load_documents</code>	factual content referenced in evals
tests/	Unit tests for core modules	e.g., <code>test_chunk_documents</code> , <code>test_trace_summary</code>	validates chunk <code>encoder</code> , or top-hit, telemetry summary shape

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>artifacts/</code>	Persisted outputs from pipeline runs	eval summaries, predictions, traces, reports	index/eval/report/telemetry output files

Module 3: Core Execution Flows

3.1 Flow A: Build Index (`rag-telemetry-evals build-index`)

Entrypoint chain:

1. `cli.py::build_index_cmd()`
2. `pipeline.py::build_index(settings, gateway, tracer)`

Step-by-step:

1. Validate local Ollama models:
 - `await gateway.ensure_required_models(settings.chat_model, settings.embedding_model)`
2. Load source docs:
 - `load_documents(settings.resolved_knowledge_dir)`
 - Filters to `.md` and `.txt`, skips empty files.
3. Chunk docs:
 - `chunk_documents(..., chunk_size_words, chunk_overlap_words)`
 - Each chunk is `DocumentChunk` with `chunk_id`, `source`, `title`, `text`, `start_word`, `end_word`.
4. Embed chunks:
 - `gateway.embed_texts(model=settings.embedding_model, texts=[...], timeout_seconds=settings.embedding_timeout_seconds)`
 - Returns `np.ndarray` shape `[n_chunks, embedding_dim]`.
5. Build + persist index:
 - `build_vector_index(chunks, embeddings)` checks alignment.
 - `save_vector_index(...)` writes:
 - `artifacts/index/embeddings.npy`
 - `artifacts/index/chunks.json`
6. Return payload shape:

```
{
  "n_documents": 5,
  "n_chunks": 5,
  "embedding_dimension": 768,
```

```

    "embeddings_file": ".../artifacts/index/embeddings.npy",
    "chunks_file": ".../artifacts/index/chunks.json"
}

```

Telemetry spans emitted in this flow: `load_documents`, `chunk_documents`, `embed_chunks`, `persist_index`.

3.2 Flow B: Ask Question (RAG vs Baseline)

Entrypoint chain:

1. `cli.py::ask_cmd(question, rag)`
2. `pipeline.py::answer_question(settings, question, use_rag)`

Common setup:

- Validate models via `ensure_required_models`.
- Load persisted index with `load_index(settings)`.
- Construct `LocalRAGEngine(settings, gateway, index, tracer)`.
- Trace IDs are generated as `ask-<epoch_ms>`.

B1. Baseline path (`--baseline`)

- Calls `LocalRAGEngine.answer_without_rag(question, trace_id)`.
- Generates with `BASELINE_SYSTEM_PROMPT` and user question only.
- Timeout fallback: `ChatResult(text="I do not know.", done_reason="timeout_fallback")`.

Output shape:

```

{
  "mode": "baseline",
  "trace_id": "ask-...",
  "question": "...",
  "answer": "...",
  "metadata": {
    "text": "...",
    "prompt_tokens": 0,
    "completion_tokens": 0,
    "total_duration_ns": 0,
    "prompt_eval_duration_ns": 0,
    "eval_duration_ns": 0,
    "load_duration_ns": 0,
    "done_reason": "timeout_fallback"
  },
  "retrieved": []
}

```

B2. RAG path (`--rag`)

- Calls `LocalRAGEngine.answer_with_rag(question, trace_id)`.

- Retrieval stage (`retrieve`):
 - Primary: embed question + `VectorIndex.search(...)`.
 - Fallback on embedding timeout: `_keyword_fallback(question)` using token overlap.
- Builds context block from retrieved chunks (`source`, `score`, `text`).
- Generation uses `RAG_SYSTEM_PROMPT` and a structured user prompt requesting 2-4 sentences + source filenames.
- Timeout fallback: `_extractive_fallback(retrieved)`.

retrieved list is normalized in `pipeline._retrieval_to_records(...)`:

```
{
  "score": round(item.score, 4),
  "source": item.chunk.source,
  "chunk_id": item.chunk.chunk_id,
  "text": item.chunk.text,
}
```

RAG output payload adds those retrieval records in `retrieved`.

Telemetry spans here include: `answer_rag` and nested `retrieve`, `generate_rag`.

3.3 Flow C: Evaluation (`rag-telemetry-evals evaluate`)

Entrypoint chain:

1. `cli.py::evaluate_cmd()`
2. `pipeline.py::evaluate(settings, reset_trace_file=True)`
3. `eval/evaluator.py::run_evaluation(...)`

Step-by-step:

1. Optional trace reset (`traces.jsonl` unlinked if requested).
2. Load eval dataset: `load_eval_examples(data/eval/questions.json)`
-> list of `QAExample`.
3. For each question:
 - baseline answer (`answer_without_rag`)
 - rag answer + retrieved chunks (`answer_with_rag`)
 - compute:
 - keyword recall (`keyword_recall`)
 - semantic similarity (`_semantic_similarity`)
 - retrieval hit (`expected_source` in retrieved sources)
 - source mention rate (`expected_source` substring in rag answer)
4. Aggregate into `EvalSummary` means/gains.
5. Persist outputs:
 - `save_predictions(rows, artifacts/evals/predictions.csv)`
 - `save_summary(summary, artifacts/evals/summary.json)`
 - `summarize_traces(..., artifacts/telemetry/summary.json)`

- `render_report(..., artifacts/reports/rag-telemetry_eval_report.md)`

Prediction CSV columns (from EvalPrediction):

- `question_id`, `question`, `expected_source`, `reference_answer`
- `baseline_answer`, `rag_answer`
- `baseline_keyword_recall`, `rag_keyword_recall`
- `baseline_semantic_similarity`, `rag_semantic_similarity`
- `retrieval_hit`, `rag_mentions_expected_source`
- `baseline_latency_ms`, `rag_latency_ms`
- `baseline_prompt_tokens`, `rag_prompt_tokens`
- `baseline_completion_tokens`, `rag_completion_tokens`

Summary JSON keys (from EvalSummary):

- `n_questions`
- `retrieval_hit_rate`
- `rag_mentions_expected_source_rate`
- `baseline_keyword_recall_mean`
- `rag_keyword_recall_mean`
- `keyword_recall_gain`
- `baseline_semantic_similarity_mean`
- `rag_semantic_similarity_mean`
- `semantic_similarity_gain`
- `baseline_latency_ms_mean`
- `rag_latency_ms_mean`
- `baseline_total_tokens_mean`
- `rag_total_tokens_mean`

3.4 Flow D: Full run (`rag-telemetry-evals run-all`)

Entrypoint chain:

1. `cli.py::run_all_cmd()`
2. `pipeline.py::run_all(settings)`

Behavior:

- Deletes old `traces.jsonl` and telemetry summary if present.
- Runs `build_index(...)` then `evaluate(..., reset_trace_file=False)`.
- Writes `artifacts/run_summary.json` containing:
 - selected chat/embedding model names
 - index payload
 - evaluation payload
 - run summary output path

3.5 Flow E: Telemetry-only summary (`rag-telemetry-evals summarize-telemetry`)

Entrypoint chain:

1. `cli.py::summarize_telemetry_cmd()`
2. `telemetry/tracer.py::summarize_traces(trace_file, telemetry_summary_file)`

Input:

- `artifacts/telemetry/traces.jsonl` where each line matches `TraceSpanRecord`:
 - `trace_id`, `span_name`, `status`, `start_time_utc`, `end_time_utc`, `latency_ms`, `attributes`

Output summary shape:

```
{
  "generated_at_utc": "...",
  "trace_file": ".../traces.jsonl",
  "n_spans": 40,
  "n_unique_traces": 16,
  "by_span": [
    {
      "span_name": "retrieve",
      "count": 8,
      "error_count": 0,
      "latency_ms_mean": 10439.702,
      "latency_ms_p50": 12007.702,
      "latency_ms_p95": 12014.285,
      "latency_ms_max": 12014.478
    }
  ]
}
```

Module 4: Setup & Run Guide

4.1 Prerequisites

- Linux/macOS shell.
- Python 3.12.10 (see `.python-version` and `pyproject.toml`).
- `uv` package manager.
- Running local Ollama server (`OLLAMA_HOST`, default `http://127.0.0.1:11434`).

4.2 Install on a clean machine

```
git clone https://github.com/pypi-ahmad/rag-telemetry-evals-ollama.git
cd rag-telemetry-evals-ollama
uv python pin 3.12.10
uv sync --dev
cp .env.example .env
```

Pull required models:

```
ollama pull embeddinggemma:latest
ollama pull phi3.5:3.8b
```

4.3 Environment variables (.env)

Required/used keys (from .env.example and Settings):

- OLLAMA_HOST
- CHAT_MODEL
- EMBEDDING_MODEL
- SEED
- CHUNK_SIZE_WORDS
- CHUNK_OVERLAP_WORDS
- RETRIEVAL_TOP_K
- GENERATION_TEMPERATURE
- GENERATION_MAX_TOKENS
- GENERATION_TIMEOUT_SECONDS
- EMBEDDING_TIMEOUT_SECONDS
- KNOWLEDGE_DIR
- EVALUATION_FILE
- ARTIFACTS_DIR
- INDEX_DIR
- EVAL_DIR
- REPORT_DIR
- TELEMETRY_DIR

Notes:

- Paths may be relative; `Settings.resolve(...)` makes them absolute from project root.
- `get_settings()` always calls `ensure_dirs()` to create required directories.

4.4 Typical command sequences

Build index only:

```
uv run rag-telemetry-evals build-index
```

Ask one question with RAG:

```
uv run rag-telemetry-evals ask "What is the enterprise first-response SLA?" --rag
```

Ask baseline:

```
uv run rag-telemetry-evals ask "What is the enterprise first-response SLA?" --baseline
```

Run evaluation:

```
uv run rag-telemetry-evals evaluate
```

Run full pipeline:

```
uv run rag-telemetry-evals run-all
```

Summarize telemetry:

```
uv run rag-telemetry-evals summarize-telemetry
```

Notebook flow:

```
uv run python scripts/execute_notebooks.py
```

4.5 Migration/seeding/external services

- **Database migrations:** none (no DB layer in this repo).
- **Seeding:** knowledge corpus is file-based under `data/knowledge/`; evaluation set is `data/eval/questions.json`.
- **External dependency:** Ollama runtime + pulled local models.
- **Generated artifacts:** created under `artifacts/` when `index/eval/report/telemetry` commands run.

4.6 Validation and quality tooling in repository config

From `pyproject.toml` and `.github/workflows/ci.yml`:

- Lint: `uv run ruff check .`
- Type-check: `uv run mypy src`
- Tests: `uv run pytest`

Module 5: Study Plan & Practice Exercises

5.1 Suggested study order (for a new learner)

1. Read `README.md` for the project objective and command-level mental model.
2. Read `pyproject.toml` and `.env.example` to understand runtime dependencies and config surface.
3. Read `src/rag-telemetry-evals/schemas.py` to lock down data contracts first.
4. Read `src/rag-telemetry-evals/config.py` to understand path resolution and settings defaults/constraints.
5. Read `src/rag-telemetry-evals/cli.py` to map user commands to pipeline functions.
6. Read retrieval/indexing internals:
 - `data/documents.py`
 - `retrieval/chunking.py`
 - `retrieval/vector_index.py`
7. Read generation internals: `retrieval/rag_engine.py` and `ollama_client.py`.
8. Read evaluation/report/telemetry modules:
 - `eval/evaluator.py`

- `telemetry/tracer.py`
 - `reporting/reporting.py`
9. Finally inspect real outputs in `artifacts/` and match each file to its producing function.

5.2 Practice exercises

1. Entrypoint tracing

- Task: Trace exactly what happens when `rag-telemetry-evals run-all` is executed.
- Focus files: `cli.py`, `pipeline.py`.

2. Schema reconstruction

- Task: Write the field list (names + meaning) for `EvalPrediction` and `EvalSummary`.
- Focus file: `schemas.py`.

3. Retrieval fallback reasoning

- Task: Explain when retrieval uses vector similarity vs keyword fallback, and what telemetry attributes reveal which path was used.
- Focus files: `rag_engine.py`, `telemetry/tracer.py`, `artifacts/telemetry/traces.jsonl`.

4. Chunking math

- Task: Given `chunk_size_words=170` and `chunk_overlap_words=35`, compute the step size and explain why overlap exists.
- Focus file: `retrieval/chunking.py`.

5. Index artifact mapping

- Task: Identify which function writes `embeddings.npy` and which writes `chunks.json`, and explain the alignment invariant.
- Focus file: `retrieval/vector_index.py`.

6. Evaluation metric interpretation

- Task: Explain how `keyword_recall_gain` and `semantic_similarity_gain` are computed.
- Focus file: `eval/evaluator.py`.

7. Error-path comprehension

- Task: Describe baseline and RAG behavior when generation times out.
- Focus file: `rag_engine.py`.

8. Report provenance check

- Task: For one metric shown in `artifacts/reports/rag-telemetry_eval_report.md`, identify its source field in `EvalSummary` and where it is rendered.
- Focus files: `schemas.py`, `reporting/reporting.py`.

9. Config-path deep dive

- Task: Explain how `KNOWLEDGE_DIR=data/knowledge` becomes an absolute path at runtime.
- Focus file: `config.py`.

10. Telemetry aggregation logic

- Task: Explain how p95 latency is computed and where values are persisted.
- Focus file: `telemetry/tracer.py`.

5.3 Model answer outlines

1. Entrypoint tracing (outline)

- `run_all_cmd()` -> `_run_async(run_all(settings))` -> `run_all()` calls `build_index()` then `evaluate()` -> writes `artifacts/run_summary.json`.

2. Schema reconstruction (outline)

- `EvalPrediction`: per-question answers, metrics, retrieval booleans, latency, tokens.
- `EvalSummary`: aggregated means/rates/gains and mean token+latency metrics.

3. Retrieval fallback reasoning (outline)

- `retrieve()` tries embedding + vector search first.
- On `TimeoutError`, sets `retrieval_mode="keyword_fallback"` and scores chunk term overlap.
- `n_retrieved/top_score` appear in span attributes.

4. Chunking math (outline)

- `Step size = chunk_size_words - chunk_overlap_words = 135`.
- Overlap preserves local continuity between adjacent chunks for better retrieval recall.

5. Index artifact mapping (outline)

- `save_vector_index(...)` writes both files.
- `build_vector_index(...)` enforces `len(chunks) == len(embeddings)` to keep row-to-chunk alignment.

6. Evaluation metric interpretation (outline)

- Gains are simple mean differences: `rag_mean - baseline_mean` for keyword recall and semantic similarity.

7. Error-path comprehension (outline)

- Baseline timeout -> fixed answer "I do not know." with `done_reason="timeout_fallback"`.
- RAG timeout -> extractive fallback from top retrieved chunk (or unknown if none).

8. Report provenance check (outline)

- Example: `rag_semantic_similarity_mean` in `EvalSummary` becomes template variable `rag_semantic` in `render_report` and appears in markdown report.

9. Config-path deep dive (outline)

- `Settings.resolve(path)` joins relative paths with `project_root` (`Path(__file__).resolve().parents[2]`).
- `resolved_knowledge_dir` property exposes absolute path used by loaders.

10. Telemetry aggregation logic (outline)

- `summarize_traces()` groups spans by `span_name`.
- `p95` computed by `_safe_percentile(values, 0.95)` using `numpy.quantile`.
- Writes JSON summary to `settings.telemetry_summary_file`.

Understanding Checklist

Use this checklist before claiming mastery:

- ☐ Can you explain `run-all` end-to-end with exact function calls and output files?
- ☐ Can you describe the difference between `answer_with_rag` and `answer_without_rag` including timeout behavior?
- ☐ Can you list all key fields in `QAExample`, `EvalPrediction`, and `EvalSummary` without guessing?
- ☐ Can you explain how chunking overlap is implemented and why it helps retrieval?
- ☐ Can you explain how vector search scoring works in `VectorIndex.search`?
- ☐ Can you map each artifact file under `artifacts/` to the function that writes it?
- ☐ Can you read one trace line and interpret `trace_id`, `span_name`, `status`, and `attributes`?
- ☐ Can you explain how report values are transformed from summary objects into markdown output?
- ☐ Can you identify exactly where environment variables become runtime settings?
- ☐ Can you explain why this repo uses async functions and where the network boundary is?